

Алгоритмы и процессы

Снижение нагрузки на облако

При выгрузке данных реестра источником выступает как правило основной сервис. Для него критически важно распределение нагрузки по времени, чтобы не возникало длительных вызовов метода БЛ и не создавались очереди в основном и служебном пулах.

Эта задача решается следующими механизмами:

- выполнение выгрузки в сценарии LRS
- разделение процесса чтения данных и формирования страниц выгрузки на серию вызовов
- сохранение данных реестров с медленным чтением данных (менее 20 000 записей в минуту) в сессиях сервиса **report-prefetch**

По умолчанию формируется итеративный сценарий выгрузки. Длительность итерации ограничена 10 минутами, это время считается безопасным временем выполнения метода БЛ в служебном пуле. В итерациях выгрузки автоматически оценивается скорость чтения данных. Если в течение одной итерации невозможно сформировать страницу выгрузки, сценарий переключается в режим медленной выгрузки, с сохранением данных в сессиях **report-prefetch**.

Для реестров, гарантированно ограниченных по объему данных, допустимо использование неитеративного сценария выгрузки.

Неитеративный сценарий выгрузки

Для снижения нагрузки на БЛ от долгих операций выгрузки предусмотрены ограничения одновременного выполнения:

- на сценарий DWC - по пользователю
- на задачу выгрузки данных сценария - по сервису

Неитеративный сценарий включается опцией выгрузки **UseIterativeExport**, при передаче в это поле значения **False**.

Чтение шаблона выгрузки и формирование файла выгрузки выполняется в рамках этой же задачи сценария. В зависимости от объема данных, формируется **xlsx**-файл со всем набором данных либо **zip**-архив, содержащий несколько "страниц выгрузки" - **xlsx**-файлов заданного размера (по-умолчанию, по 100 000 строк).

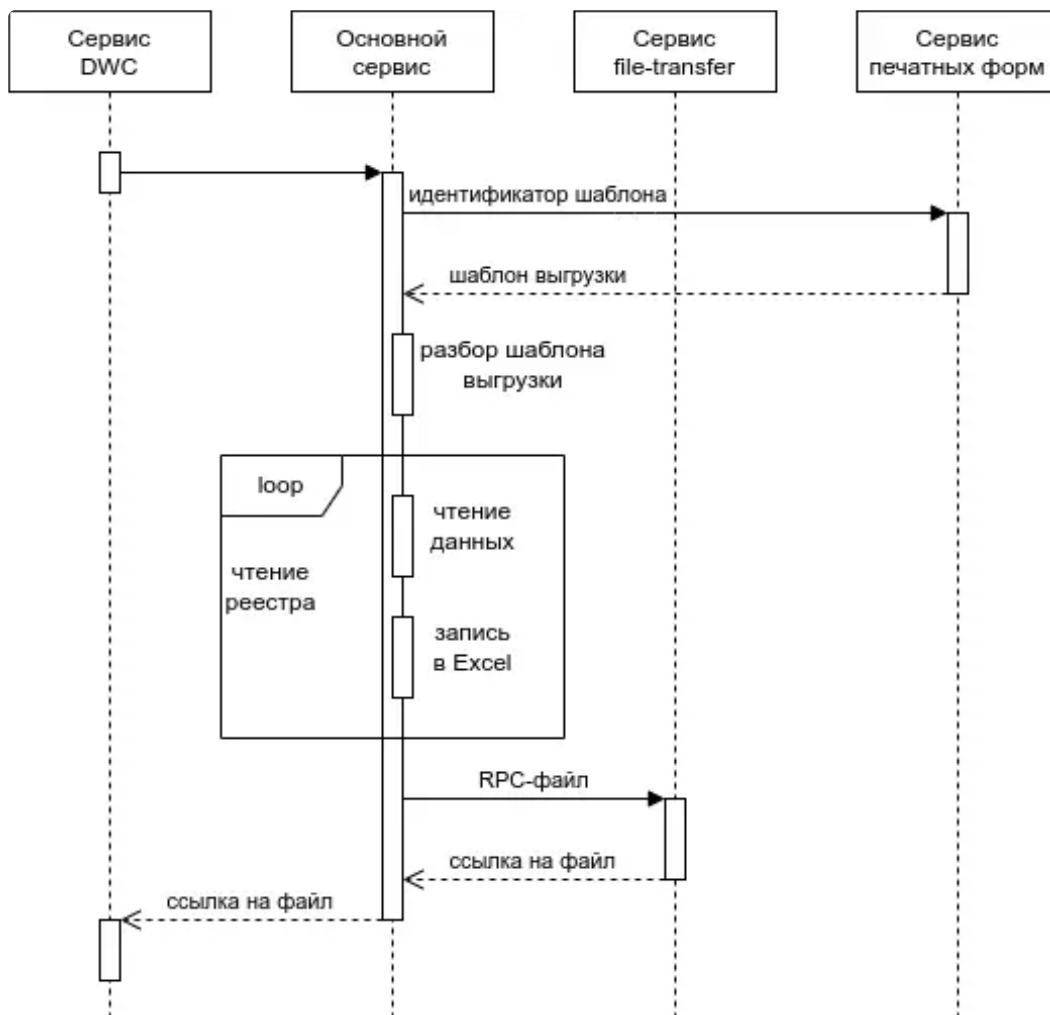


Диаграмма 1. Неитеративный сценарий выгрузки

Итеративный сценарий выгрузки

Итеративный сценарий выгрузки предусматривает выполнение следующих технических задач на сервисе **import-excel**:

- получения настроек выгрузки и оформления Excel-документ из шаблона выгрузки,
- формирования итогового файла выгрузки, в зависимости от объема данных, это xlsx-файл со всем набором данных либо zip-архив, содержащий несколько "страниц выгрузки" - xlsx-файлов заданного размера (по-умолчанию, 100 000 строк).

Таким образом, с основного сервиса снимается вся нагрузка, не связанная непосредственно с чтением данных.

Итеративный сценарий включается опцией выгрузки **UseIterativeExport**, при передаче в это поле значения **True**.

Задача получения настроек выгрузки и оформления **Excel.PrepareExportParameters** формирует настройки выгрузки и возвращает их для передачи в качестве входного параметра в итеративный метод.

Задача итеративного блока сценария DWC **Excel.SaveIterative** выполняет чтение данных выгрузки прикладным методом, страница выгрузки формируется по мере начитывания строк. После получения заданного количества строк, xlsx-файл сохраняется в хранилище file-transfer, и итерация завершается. Метод возвращает текущую навигацию для вызова чтения данных в следующей итерации и данные результатов выполнения предыдущих итераций, в том числе ссылки на сформированные страницы выгрузки.

На метод **Excel.SaveIterative** накладывается ограничение на одновременное выполнение задач выгрузки.

Задача формирования результатов выгрузки **Excel.CreateIterationsResult** получает результат выполнения последней итерации и по списку страниц выгрузки формирует zip-архив результата и ссылку для его скачивания, либо же ссылку для скачивания единственной страницы выгрузки.

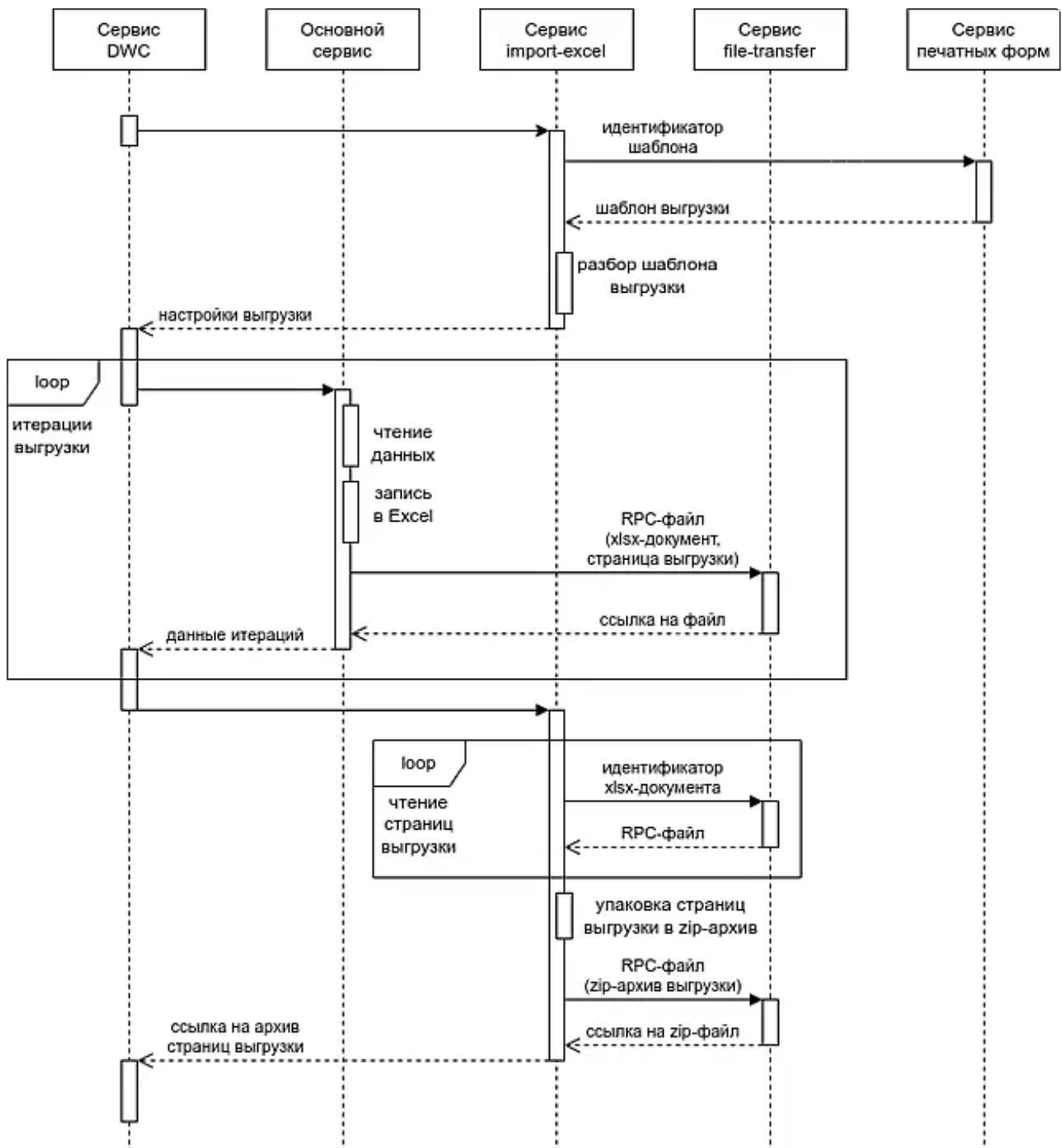


Диаграмма 2. Итеративный сценарий выгрузки

Итеративный сценарий медленной выгрузки

Для случаев, когда прикладной метод чтения данных не позволяет получить полный набор данных для страницы выгрузки за максимальное время итерации, предусмотрен альтернативный механизм временного хранения данных выгрузки. При этом выгрузка выполняется тем же сценарием, итеративный метод переключается на альтернативный механизм хранения при необходимости во время выполнения, либо по данным предыдущей итерации.

Метод **Excel.Saveltiterative** фиксирует время начала итерации и замеряет время чтения данных прикладным методом, по максимальному времени чтения строит пессимистичный прогноз времени построения страницы. Если оценка времени выполнения превышает лимит на итерацию происходит переключение: формируется сессия **report-prefetch**, полученные блоки данных и все последующие сохраняются в сессии. Итерация прерывается, если перед чтением очередного блока данных в оставшееся до лимита время не укладывается удвоенное максимальное время чтения. При этом в результате итерации передается идентификатор сессии **report-prefetch** и количество полученных строк, ссылка на страницу выгрузки в хранилище **file-transfer** остается пустой.

Если метод получил от предыдущей итерации идентификатор сессии **report-prefetch**, то выгрузка сразу продолжается в режиме хранения в сессиях. Если в предыдущем вызове не получено достаточно строк для формирования страницы выгрузки, сохранение продолжается в полученной сессии, иначе формируется новая.

Метод **Excel.CreateIterationsResult** при формировании результата выгрузки, если в данных итерации указана полная сессия **report-prefetch**, считывает данные сессии и формирует xlsx-файл страницы выгрузки. Далее страницы

упаковываются в результат выгрузки точно так же, как и при чтении готовых страниц из хранилища **file-transfer**.

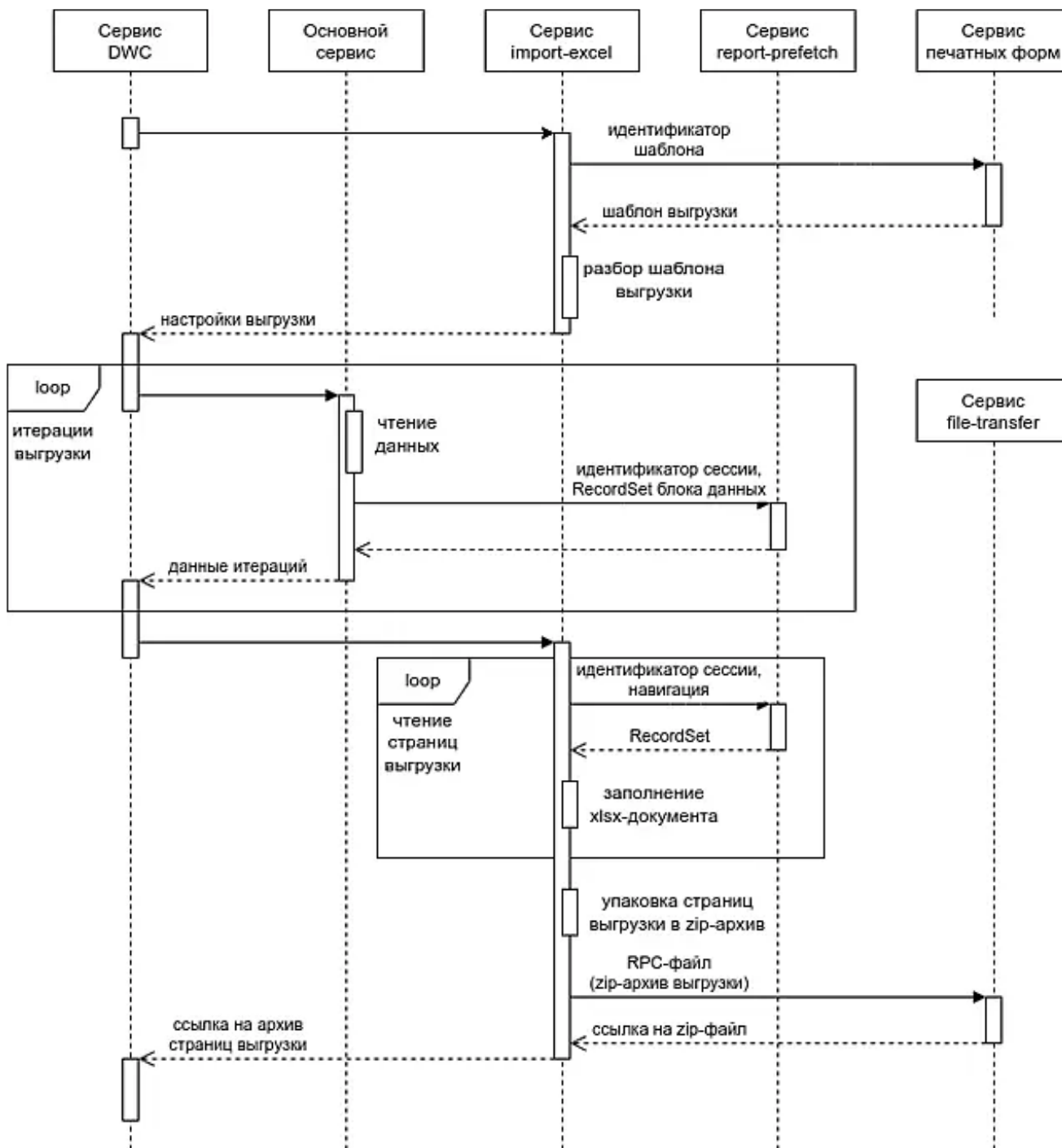


Диаграмма 3. Итеративный сценарий медленной выгрузки

Настройка шаблонов выгрузки

Шаблоны выгрузки являются **xlsx**-документами и хранятся на сервисе Печатных форм.

Список шаблонов выгрузки для клиента хранится на сервисе **parameters**, обновляется методом **Excel.SetPresets**, читается методом **Excel.GetPresets**.

Создание шаблона

1. Методом **Excel.GetPresets** читается список существующих шаблонов выгрузки
2. Вызывается метод **Excel.CreateTemplate**, метод возвращает **GUID** нового шаблона выгрузки
3. Методом **PrintingTemplate.Exists** проверяется, что шаблон существует
4. Методом **PrintingTemplate.GetById** из сервиса печатных форм получаются метаданные шаблона
5. Методом **DocView.ReadAttachmentAsPreviewSync** по хэшу содержимого получается превью документа в виде изображения

Если изменения шаблона не выполняется, при подтверждении создания вызываются методы:

1. Методом Excel.GetPresets читается список существующих шаблонов выгрузки
2. Полученный список шаблонов дополняется созданным.
3. Методом Excel.SetPresets обновляется список шаблонов на сервисе parameters

Изменение шаблона

Выполняется после изменения набора полей:

1. Вызывается метод Excel.UpdateTemplate, в Excel-файл шаблона записываются данные превью с новым набором полей.
2. Методом PrintingTemplate.Exists проверяется, что шаблон существует
3. Методом PrintingTemplate.GetByld из сервиса печатных форм получают метаданные шаблона
4. Методом DocView.ReadAttachmentAsPreviewSync по хэшу содержимого получается превью документа в виде изображения

Выполняется при открытии файла шаблона на редактирование:

1. Методом PrintingTemplate.GetByld из сервиса печатных форм получают метаданные шаблона
2. Методом PrintingTemplateFileEditor.ReadAttachmentInfo получают метаданные Excel-файла
3. PrintingTemplateFileEditor.ReadFileInfo

Выполняется после сохранения файла шаблона:

1. Методом PrintingTemplate.Exists проверяется, что шаблон существует
2. Методом PrintingTemplate.GetByld из сервиса печатных форм получают метаданные шаблона
3. Методом DocView.ReadAttachmentAsPreviewSync по хэшу содержимого получается превью документа в виде изображения

Выполняется после подтверждения изменений:

1. Вызывается метод Excel.CreateCommit для нового шаблона или Excel.UpdateCommit для существующего, публикуется событие в историю.
2. Методом Excel.GetPresets читается список существующих шаблонов выгрузки
3. Полученный список шаблонов обновляется по изменениям.
4. Методом Excel.SetPresets обновляется список шаблонов на сервисе parameters

Удаление шаблона

Удаление шаблона выгрузки из интерфейса настройки выгрузки происходит в 2 этапа:

1. Методом Excel.GetPresets читается список существующих шаблонов выгрузки
2. Методом Excel.DeleteTemplate шаблон удаляется на Сервисе печатных форм.
3. Из полученного ранее списка шаблонов исключается удаленный.
4. Методом Excel.SetPresets обновляется список шаблонов на сервисе parameters.